Last Updated: 2026-03-30 Mon 17:21

CMSC416 Assignment 3: Theoretical and Empirical Performance Analysis

- **Due: Tue Tue 07-Apr by 11:59 pm**
- *Approximately 5% of total grade*
- Submit to **Gradescope** in PDF format
- You may work in groups of 1 to 3 and submit one assignment per group.

CODE DISTRIBUTION: a2-code.zip

CHANGELOG: Empty

Table of Contents

- 1. (30pts) **Problem 1**: Analytic Performance of Parallel LU Factorization
- 2. (20pts) **Problem 2**: Basic Use of HPC Toolkit
- 3. (40pts) **Problem 3**: Empirical Analysis of Parallel LU Factorization with HPC Toolkit
- 4. (10pts) **Work Disclosure**
- 5. Submission Instructions

ASSIGNMENT WRITEUP CONVENTIONS

This is a primarily written assignment rather than a coding assignment. Please abide by the following conventions while completing the assignment.

- **Type your answers** for the most part. Use a document format convenient for you (Google Docs, Microsoft Word, Latex / Overleaf, etc.) but make sure that answers are typed in it.
- Submit a PDF of your work. Do your work in an application that allows saving PDFs that are paged (letter sized 8.5in by 11in).
- Mathematical equations and derivations may be hand-written with photos of the math included in your writeup. Typographical versions of math are preferred for readability to hand-written work so do that if you know how and make sure any hand-written math is readable.
- When submitting to Gradescope, mark pages that contain answers to each problem to ease the grading burden.

  Note: some of the math fonts used throughout the assignment may not display correctly. I have tested the display on Google Chrome and Mozilla Firefox and seen it work when connected to the network. Notify Prof Kauffman via email if it does not render properly.

1 (30pts) Problem 1: Analytic Performance of Parallel LU Factorization

We discussed the LU factorization process in lecture and how to parallelize it. Below is pseudocode for a parallel implementation of LU factorization. It has the following features.

- Assumes that the large, square matrix rows of the `A[]` matrix are evenly distributed among the processors
- Leaves modifies the A matrix in place overwriting it with the L and U factors in its Lower and Upper triangle
- Assumes that the processors are arranged in a 1D torus so that broadcasting requires: $T_{broadcast} = t_s \log_2(P) + t_w PM$ with P as the number of processors and M
- Assumes that all processors do (approximately) the same number of operations including broadcasting. A more sophisticated version would potentially cut processors out of broadcasts / computations as they finish before others.

```
PARALLEL_LU_FACTORS_INPLACE(A[] : square matrix,
                            proc_count : number of processors)
{
    // Will overwrite A with its L*U factors, no allocation of L or U
    // Assumes rows of A[] are distributed among all processors
    N = nrows(A)
    row_count = N / proc_count                  // assume even division for simplicity
    for(d=0 to N-1){                            // d is the leading row index
        Proc with row A[d,:] Broadcasts row to all others as Arow_d
        for(row Arow_i in A[] owned by Proc){   // each proc updates its row_count rows of
            if(Arow_i row index > d){           // as algorithm advances, low-index rows do
                scale = Arow_i[d] / Arow_d[d]   // scaling factor for this row to 0 out lea
```

```
                Arow_i[d] = scale                // store scle factors in the Lower triangle
                for(j=d+1 to N-1){               // adjust row by scaled lead row
                    Arow_i[j] = Arow_[j] - scale*Arow_d[j]
                }
            }
        }
    }
    return; // A now has its L,U factors in its lower/upper triangles
}
```

Do the following analysis with this code.

1. **Parallel Runtime:** Computer the Parallel Runtime T_{par} for the algorithm. Use the following variables in your analysis.
   - N for the matrix dimension (A is an $N \times N$ square matrix)
   - P for the number of processors
   - t_s and t_w for communication startup and per-word transfer times
   - The given time complexity for the Broadcast operation for a 1D Torus
2. **Parallel Overhead:** Compute the Parallel Overhead of the algorithm which is $T_{over} = PT_{par} - T_{ser}$. The Serial runtime of the in-place LU factorization is N^3.
3. **Isoefficiency Balance Equation:** Provide the Isoefficiency Balance equation for the Parallel LU Factorization by substituting terms into the equation $T_{ser} = KT_{over}$ as discussed in class. Remember K is an arbitrary constant indicating the desired efficiency so is not filled in.
   - Simplify your equation as much as possible so that one side is only N and the other side is an expression of constants, P, and possibly an N in the denominator of a term.
   - Describe how this equation governs how to maintain efficiency when increasing P.
   - If going from $P = 64$ procs to $P = 128$ procs, how much should N increase so that efficiency stays the same?
   - For $N = 1024, P = 64, t_s = 1, t_w = 0.1$ compute the Efficiency of the Parallel Algorithm which is $E = T_{ser}/(PT_{par}$.
   - Then for $N = ??, P = 128, t_s = 1, t_w = 0.1$, compute Efficiency again. Fill in the ?? with the value for N predicted by Isoefficiency to show it correctly predicts how N needs to scale with P to maintain the same efficiency level.

2 (20pts) Problem 2: Basic Use of HPC Toolkit

This problem will have get students familiar with running HPC Toolkit to profile a parallel code and analyze it's results. The steps below will guide one through using HPC Toolkit on Zaratan to profile a code. Some steps ask that students provide screenshots of a Terminal output to verify that they are tracking their steps. Consult your Operating System of choice on how to take screenshots (Windows: Snipping Tool, MacOS Screenshot Tool, Linux: Various).

This problem will require installing some software to analyze the data so ensure you have ready access to such a machine.

While there are a lot of steps here, it should take less than an hour to complete this problem.

1. We will profile the Heat simulation from Assignment 2. Students that did not finish the Heat implementation from Assignment 2, either partner with someone who did or approach Professor Kauffman to get access to an implementation.
2. Log into Zaratan via SSH and change into the directory where the Heat solution is located (likely `a2-code` or something similar). Compile the program to ensure it is ready, probably by typing `make clean` and then `make`.
3. Run the command:

   ```
   >> module load hpctoolkit
   ```

   This will put the HPC Toolkit profiling tools on the path. **NOTE: This will screw with compilation** as HPC toolkit tools are built with slightly different compiler toolchain. Load the module ONLY after compiling your own code. If in doubt, you can `module unload hpctoolkit` to get restore your path.

4. Start an interactive job run using the command

   ```
   >> sinteractive -c 8 -t 20 -a cmsc416-class
   # -c 8   : 8 cores
   ```

```
# -t 20  : 20 minutes
# -a ... : select class job allocation
```

This will pause for a moment to secure compute resources and then log your shell into a compute node to work interactively there. **Do this to abide by the "no MPI jobs on login nodes"** admin policy.

This will print messages and change the prompt according to the compute node granted. An example

```
>> sinteractive -c 8 -t 20 -a cmsc416-class
salloc: Granted job allocation 18732721
salloc: Nodes compute-b7-35 are ready for job
profk@compute-b7-35 [a2-code]%
```

5. Run the Heat MPI code under HPC Toolkit to profile and trace it. Use this command:

```
>> mpirun -np 8 hpcrun -t ./heat_mpi 50000 20000 0
# -np 8 : run with 8 processors
# -t    : trace the run to show activity over time
# 50000 : time steps
# 20000 : number of heat "bar" elements evenly distributed amont procs
# 0     : do not print the final output
```

6. Verify that HPC Toolkit has taken measurements by listing the directory where they will be stored via the command.

```
>> ls -d *heat_mpi-measure*
# should show a directory like hpctoolkit-heat_mpi-measurements-1234567
```

The "Measurements" directory requires additional processing by HPC Toolkit programs (next step).

7. Process the measurements and heat executable by running the following commands which the list the database directory created.

```
>> hpcstruct hpctoolkit-heat_mpi-measurements-NNNNNNNN
>> hpcprof   hpctoolkit-heat_mpi-measurements-NNNNNNNN
#                                            ^^^^^^^^
# Substitute the number of your run here, likely via tab completion in the shell

>> ls -d *heat_mpi-database*
# should list something like hpctoolkit-heat_mpi-database-1234567
```

This "Database" directory will be transferred and analyzed in the next steps.

8. **Take a Screenshot of your Terminal run** which shows the session of creating and interactive job, running Heat under MPI/HPC Toolkit, and post-processing the data described in the above steps.

   As you would wish to analyze other program runs, these are the steps to go through to create the dataset of observations.

9. Transfer the data in the `hpctoolkit-heat_mpi-database-*` directory to your local machine (laptop / desktop). A quick way to do so is via the `scp` utility Open a terminal on your Laptop (Windows `cmd.exe` / MacOS Terminal.app / Linux Various). Type the command

```
homeputer>> cd your/favorite/folder
# change to a local folder you want the data in

homeputer>> scp -r  USERID@login.zaratan.umd.edu:path/to/database/hpctoolkit-heat_mpi-d
```

```
#                   ^^^^^^                    ^^^^^^^^^^^^^^^^^
# Adjust            THIS                      AND THIS
```

This will prompt for login credentials and then copy the Database directory from Zaratan to your home directory.

If you are using VS Codium + SFTP Plugin, then you can do the above via a "Sync Remote -> Local".

10. After transferring the Database directory, open that folder and **Take a screenshot of the Database directory contents on your local computer**. Include the screenshot in the assignment writeup.

11. Download and install the HPCViewer program which is part of the HPC Toolkit project.

    - Download from here: https://hpctoolkit.org/download.html
    - MacOS Users: check your systems settings to determine if you have a newer M1/M2/M3 (Arm/aarch64) processor OR instead an older x86-64 processor and select an appropriate choice.

    After downloading the program, start it which should create a blank window.

12. Within HPC Viewer, select *File -> Open Local Database*. Navigate to the location where you downloaded the `hpctoolkit-heat_mpi-database-*` and select that folder (don't open the folder, just pick the folder then press the "Open" button in the dialogue). This should open the database and show several panels of information.

13. At the top of the HPC Viewer window, there will be a *Profile Tab* and a *Trace Tab*. **Start with the Profile Tab** which should be open by default. In the Bottom panel there will Foldable items starting with "program root" and "main": expand these to show the portions of code that have been identified as heavy CPU usage. There are no specific requirements for reporting here but poke around for things like

    - What percent of time was spent in which loops
    - Which function calls (MPI calls) occupied large fractions of time

    Take at least one screenshot of the Profile Tab with profiles expanded to show some features of where code spent time. In a sentence or two, mention anything interesting you found here such as the relative time spent crunching numbers versus communicating.

    (Optional) You can explore the profile options like "Top Down" (default), "Bottom Up" and "Flat" if you like which can make it quicker to identify hot spots to optimize.

14. **Switch to the Trace Tab** at the top of the HPC Viewer. This shows program activity over time color bars in the top panel. Color corresponds to to which functions were executing on which processors with the whole stack trace taken into account.
    - Time advances from left (early) to right (later) in this chart
    - There should be 8 horizontal bars for the 8 processors that ran with the color in the bar changing color from left to right as time advances and different functions are active.
    - Use the mouse "cross hair" to click on spots to show what the Function Call Stack for a processor looks like at a given moment
    - One can also "click and drag" to zoom in on a region
    - In the upper right is a "Depth: N" option which allows limiting the depth of function calls that are colored. **Set Depth to 2** since we are mostly interested here in calls out of `main()` into functions like `malloc() / MPI_Sendrecv() / MPI_Gather()`.
    - In the lower right is a "Mini Map" which allows changing where one is zoomed in on or adjusting the portion viewed

15. **Take 3 Screenshots of the Trace Window** all with the Depth set to 2

    1. The Overall run zoomed out to show all processors for the entire run
    2. Zoomed in on the first 1.0 second or so showing all cores
    3. Zoomed in on the final 1.0 second or so showing all cores

    Describe in a few sentences anything interesting revealed by the the Trace view.

    (Optional) The bottom panel shows additional information such as the function call depth of processors. You can explore what's there in the HPC Viewer Manual here:

    https://hpctoolkit.gitlab.io/hpctoolkit/users/hpcviewer/hpcviewer.html

In summary, there should be at a minimum the following screenshots with brief commentary on each in the writeup.

1. Terminal Session generating measurements / database
2. Database folder on local machine after transfer
3. HPC Viewer Profile Tab with some functions expanded
4. HPC Viewer Trace Tab with entire run, Depth: 2
5. HPC Viewer Trace Tab zoomed in on First 1.0 sec Depth: 2

6. HPC Viewer Trace Tab zoomed in on Last 1.0 sec Depth: 2

3 (40pts) Problem 3: Empirical Analysis of Parallel LU Factorization with HPC Toolkit

While the description of this problem is short, it is the longest problem so plan accordingly. Finish Problem 2 first to get familiar with HPC Viewer as it is needed

The codepack for this assignment (`a3-code.zip` linked at the top of this document) contains contains several (crummy) implementations of LU factorization along with some data on running the parallel versions of it.

- `lu_gen_random.c` : a code that generates random matrices for factorization and stores them in a binary file format read by the remaining problems.
- `lu_serial.c` : serial LU implementation
- `lu_mpi1.c` : parallel LU implementation
- `lu_mpi2.c` : parallel LU implementation
- `hpctoolkit-lu_mpi1-database-mat_10K`: HPC Toolkit database for `lu_mpi1` run on a 10K by 10K matrix
- `hpctoolkit-lu_mpi2-database-mat_10K`: HPC Toolkit database for `lu_mpi2` run on a 10K by 10K matrix

This problem will analyze the practical performance implications of the code decisions in the two LU MPI implementations. The goal is to identify problems with the performance in each of them using HPC Toolkit's measurements and better understand the algorithmic choices.

- Study the code provided and come to understand it. Ensure a good understanding of the serial algorithm and the format of the matrix when stored on disk.
- Contrast the approaches taken in the MPI1 and MPI2 versions such as how the matrix is loaded from a binary file and distributed among processors. Relate this to how the main computation loop runs.
- Use HPC Viewer to open up the databases of measurements on MPI1 and MPI2 versions. Use the Trace View to look at how the computation evolved over time and what processor were doing early and later.
- Form a recommendation on an improved version of the parallel code that combines the best features of both (but don't code it).

Answer the following questions about the MPI1 and MPI2 implementations and their performance data.

1. Using HPC Viewer, determine how many processors were used in the evaluation runs of both codes and the overall runtimes for each of MPI1 and MPI2. Which appears faster and by how much?
2. Study the codes and contrast the approaches that MPI1 and MPI2 take to distributing data between processors. Describe each approach in a few sentences using terminology discussed in the course for matrix layout. Include whether the overall data distribution is equal or unbalanced between processors in both MPI1 and MPI2.
3. Analyze how the data files are processed initially in both MPI1 and MPI2 to distribute data from the file to processors. Describe briefly how the code accomplishes these tasks focusing on the differences between the MPI1 and MPI2 versions.

4. Use HPC Viewer to Zoom in on the early stages of the program run for both MPI1 and MPI2. Which of the two appears to have a more effective strategy for loading/distributing data and gets started on computation sooner? Justify your answer by providing a screenshot of what HPC Viewer shows that demonstrates empirical performance at startup. **Include screenshots from HPC Viewer and written discussion of what you see there.**

   Hint: Loading the matrix data happens early in the runs.

5. Analyze the overall processor usage. Do both MPI1 and MPI2 have most processors executing computation for most of the their run or is one more effective than the other at this? Tie this to the code analysis and **include screenshots from HPC Viewer and written discussion of what you see there** to support your case.

   Hint: Looking at the later part of the run for processors in computation loops rather than communication is a good idea.

6. If one were producing an optimized `lu_mpi3.c` that combines the best features of the MPI1 and MPI2 versions, what would you recommend from each version?

4 (10pts) Work Disclosure

Include at the end of your document a section titled "Work Disclosure" which has sections and content as indicated in the template below.

WORK DISCLOSURE

(A) **Human Collaborators:** Aside from the 1-2 people submitting this assignment, the following people contributed ideas and discussion to the completion of this work INCLUDING course staff members. (Write NONE if no collaborators were involved.)

- Person 1 <person1@email.com> helped understand Problem X and the meaning of …

- Person 2 <person2@email.com> helped debug Cod for Problem Y…
- etc.

(B) **Resource Utilization:** The following resources such as websites, course notes, artificial intelligence tools (LLMs/ChatBots/etc.) were utilized in the completion of this work. Include course materials such as textbooks and lecture slides as well. (Write NONE if no resources were used [which would be hard to believe]).

- Resource 1 is here https://some.resource.org/useful_stuff.html and provided help for Problem Z to understand ….
- Resource 2 is the book "Parallel Analysis" by Auth Or with chapter 8 helping a lot with Problem W
- Resource 3 is here https://airegurgitator.com and provided AI refinements for the writing of problem Q and helped debug code for Problem N.
- etc.

(C) **Ownership of Work**: I / We affirm that all parties listed below have contributed to each solution presented in this document. All parties are capable of describing how the solutions were derived, how they apply to the problem, and that they were created in accordance with the course's PRIME DIRECTIVE.

Signed,

NAME(S) OF SUBMITTERS

Completing this section with reasonable thoroughness will earn full credit. Failure to include this section may result in additional penalties aside from the loss of points indicated.

5 Submission Instructions

- Upload your PDF answers to Gradescope
- Ensure your answers are typed except for screenshots and possibly mathematical equations
- Refer to A1 Submission Instructions for additional details

Web Accessibility
Author: Chris Kauffman (profk@umd.edu)
Date: 2026-03-30 Mon 17:21